

Deep Dive: 20. Advanced: Decorators

Concept Summary

Decorators wrap another function to extend its behavior without modifying the original function's code. Used heavily in web frameworks.

Coding Structure

```
@decorator_name  
def func():
```

Example Code

```
def announcer(func):  
    def wrapper():  
        print('Starting...')  
        func()  
        print('Done!')  
    return wrapper  
  
@announcer  
def task():  
    print('Working hard')  
  
task()
```

Code Explanation

- Line 1-6: We define the decorator. It creates a 'wrapper' function that executes code before and after the original function (func()).
- Line 8: We apply the decorator using the @ symbol.
- Line 12: When we call task(), we are actually calling the wrapper(). It prints 'Starting...', then 'Working hard', then 'Done!'.

Practice Problems

1. Create a decorator that converts the returned string of a function to all uppercase.
2. Create a timer decorator that calculates how long a function takes to run using the 'time' module.